

Producer Consumer Problem using Semaphore – using PThread

1. *pthread_create*: used to create a new thread

Syntax:

```
int pthread_create(pthread_t * thread,
                  const pthread_attr_t * attr,
                  void * (*start_routine)(void *),
                  void *arg);
```

Parameters:

- **thread:** pointer to an unsigned integer value that returns the thread id of the thread created.
- **attr:** pointer to a structure that is used to define thread attributes like detached state, scheduling policy, stack address, etc. Set to NULL for default thread attributes.
- **start_routine:** pointer to a subroutine that is executed by the thread. The return type and parameter type of the subroutine must be of type void *. The function has a single attribute but if multiple values need to be passed to the function, a struct must be used.
- **arg:** pointer to void that contains the arguments to the function defined in the earlier argument

2. *pthread_join*: used to wait for the termination of a thread.

Syntax:

```
int pthread_join(pthread_t th,
                 void **thread_return);
```

Parameter: This method accepts following parameters:

- **th:** thread id of the thread for which the current thread waits.
- **thread_return:** pointer to the location where the exit status of the thread mentioned in th is stored.

The POSIX system in Linux presents its own built-in semaphore library. To use it, we have to :

1. Include semaphore.h
2. Compile the code by linking with -lpthread -lrt

To lock a semaphore or wait we can use the **sem_wait** function:

```
int sem_wait(sem_t *sem);
```

To release or signal a semaphore, we use the **sem_post** function:

```
int sem_post(sem_t *sem);
```

A semaphore is initialised by using **sem_init**(for processes or threads) or **sem_open** (for IPC).

```
sem_init(sem_t *sem, int pshared, unsigned int value);
```

Where,

- **sem** : Specifies the semaphore to be initialized.
- **pshared** : This argument specifies whether or not the newly initialized semaphore is shared between processes or between threads. A non-zero value means the semaphore is shared between processes and a value of zero means it is shared between threads.
- **value** : Specifies the value to assign to the newly initialized semaphore.

To destroy a semaphore, we can use **sem_destroy**.

```
sem_destroy(sem_t *mutex);
```

To declare a semaphore, the data type is sem_t.

Aim: Program purpose to implement Producer –consumer using semaphores with PTHREADS

Program :

```
#include<unistd.h>
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
int buf[5],f,r;

sem_t mutex, full, empty;

void *produce(void *arg)
{
    int i;
    for(i=0;i<10;i++)
    {
        sem_wait(&empty);
        sem_wait(&mutex);

        printf("produced item is %d\n",i);
        buf[(++r)%5]=i;
        sleep(1);

        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consume(void *arg)
{
    int item,i;
    for(i=0;i<10;i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);

        item=buf[(++f)%5];
        printf("consumed item is %d\n",item);
        sleep(1);

        sem_post(&mutex);
        sem_post(&empty);
    }
}

int main()
{
    pthread_t tid1,tid2;

    sem_init(&mutex,0,1);
    sem_init(&full,0,0);
    sem_init(&empty,0,5);

    pthread_create(&tid1,NULL,produce,NULL);
    pthread_create(&tid2,NULL,consume,NULL);
    pthread_join(tid1,NULL);
    pthread_join(tid2,NULL);

    return 0;
```

```
}
```

Aim:Program purpose to implement Producer –consumer using semaphores

Program 2:

```
#include<stdio.h>
#include<stdlib.h>
```

```
int mutex=1,full=0,empty=3,x=0;
```

```
int main()
```

```
{
    int n;
    void producer();
    void consumer();
    int wait(int);
    int signal(int);
    printf("\n1.Producer\n2.Consumer\n3.Exit");
    while(1)
    {
        printf("\nEnter your choice:");
        scanf("%d",&n);
        switch(n)
        {
            case 1:  if((mutex==1)&&(empty!=0))
                        producer();
                    else
                        printf("Buffer is full!!");
                    break;
            case 2:  if((mutex==1)&&(full!=0))
                        consumer();
                    else
                        printf("Buffer is empty!!");
                    break;
            case 3:
                        exit(0);
                        break;
        }
    }
    return 0;
}
```

```
int wait(int s)
```

```
{
    return (--s);
}
```

```
int signal(int s)
```

```
{
    return(++s);
}
```

```
void producer()
```

```
{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;
    printf("\nProducer produces the item %d",x);
    mutex=signal(mutex);
}
```

```
void consumer()
```

```
{
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
    printf("\nConsumer consumes item %d",x);
    x--;
    mutex=signal(mutex);
}
```

Write a C Program to Simulate Paging Technique of memory management

AIM: Simulate Paging Technique of memory management.

Program:

```
#include<stdio.h>
#define MAX 50
int main()
{
    int page[MAX],i,n,f,ps,off,pno;
    int choice=0;
    printf("\nEnter the no of  pages in memory: ");
    scanf("%d",&n);
    printf("\nEnter page size: ");
    scanf("%d",&ps);
    printf("\nEnter no of frames: ");
    scanf("%d",&f);
    for(i=0;i<n;i++)
        page[i]=-1;
    printf("\nEnter the page table\n");
    printf("(Enter frame no as -1 if that page is not present in any frame)\n\n");
    printf("\npageno\tframeno\n-----\t-----");
    for(i=0;i<n;i++)
    {printf("\n\n%d\t\t",i);
    scanf("%d",&page[i]);
    }
    do
    {printf("\n\nEnter the logical address(i.e,page no & offset):");
    scanf("%d%d",&pno,&off);
    if(page[pno]==-1)
        printf("\n\nThe required page is not available in any of frames");
    else
        printf("\n\nPhysical address(i.e,frame no & offset):%d,%d",page[pno],off);
    printf("\nDo you want to continue(1/0)?");
    scanf("%d",&choice);
    }while(choice==1);
    return 1;
}
```

OUTPUT:

Enter the no of pages in memory: 3
Enter page size: 10
Enter no of frames: 5
Enter the page table
(Enter frame no as -1 if that page is not present in any frame)

pageno frameno

0	5
1	-1
2	3

Enter the logical address(i.e,page no & offset):2 500
Physical address(i.e,frame no & offset):3,500
Do you want to continue(1/0?):0

Write a C Program to simulate Segmentation technique of memory management.

AIM: Program purpose is to Implement Segmentation technique

Description: Segmentation is a memory-management scheme that supports this user view of memory. A logical address space is a collection of segments. Each segment has a name and a length. The addresses specify both the segment name and the offset within the segment. The user therefore specifies each address by two quantities: a segment name and an offset.

Segment table contains mainly two information about segment:

Base: It is the base address of the segment

Limit: It is the length of the segment.

The Segment number is mapped to the segment table. The limit of the respective segment is compared with the offset. If the offset is less than the limit then the address is valid otherwise it throws an error as the address is invalid. In the case of valid address, the base address of the segment is added to the offset to get the physical address of actual word in the main memory.

Algorithm:

Step 1: Read number of segments

Step 2: Read Segmentation Table data(Base value and Limit)

Step 3: Read Logical address(Segment no and offset value)

Step 4: With segment no as index into Segmentation Table find Limit value and base value

Step 5: Compare Limit value with offset value

1. If offset value is valid find physical address

Physical address= Base address + offset

Else print error message

Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
    int segTable[10][2], NumSeg, offset, SegNo, phyAdd;
    int i;

    printf("Enter the number of Segments of process 1\n");
    scanf("%d", &NumSeg);

    for (i = 0; i < NumSeg; i++)
    {
        printf("Enter the Size(Limit value) and BaseAddress of the segment %d:\n", i);
        scanf("%d%d", &segTable[i][0], &segTable[i][1]);
    }

    printf("Following is the segment Table entered by the user\n");
    printf("SegNo\tLimit value\tBase Address\n");
    for (i = 0; i < NumSeg; i++)
    {
        printf("%d\t%d\t%d\n", i, segTable[i][0], segTable[i][1]);
    }

    while (1)
    {
        printf("Enter the Logical Address : (Segment No and Offset value)\n");
        scanf("%d%d", &SegNo, &offset);

        if (SegNo >= NumSeg || SegNo < 0)
```

```

    {
        printf("Wrong segment number so Address binding not possible\n");
    }
    else if (offset >= segTable[SegNo][0] || offset < 0)
    {
        printf("Offset value is greater than or equal to the limit, so Address binding not possible\n");
    }
    else
    {
        phyAdd = segTable[SegNo][1] + offset;
        printf("Base Address for the given Segment Number is %d\n", segTable[SegNo][1]);
        printf("Physical address for the given Logical Address is %d\n", phyAdd);
    }
}

return 0;
}

```

OUTPUT:

Enter the number of Segments of process 1

5

Enter the Size(Limit value) and BaseAddress of the segment 0:

600 219

Enter the Size(Limit value) and BaseAddress of the segment 1:

14 2300

Enter the Size(Limit value) and BaseAddress of the segment 2:

100 90

Enter the Size(Limit value) and BaseAddress of the segment 3:

580 1327

Enter the Size(Limit value) and BaseAddress of the segment 4:

96 1952

Following is the segment Table entered by the user

SegNo	Limit value	Base Address
0	600	219
1	14	2300
2	100	90
3	580	1327
4	96	1952

Enter the Logical Address : (Segment No and Offset value)

0 430

Base Address for the given Segment Number is 219

Physical address for the given Logical Address is 649

Enter the Logical Address : (Segment No and Offset value)

1 10

Base Address for the given Segment Number is 2300

Physical address for the given Logical Address is 2310

Enter the Logical Address : (Segment No and Offset value)

2 500

Offset value is greater than or equal to the limit, so Address binding not possible

Enter the Logical Address : (Segment No and Offset value)

3 400

Base Address for the given Segment Number is 1327

Physical address for the given Logical Address is 1727

Enter the Logical Address : (Segment No and Offset value)

4 112

Offset value is greater than or equal to the limit, so Address binding not possible